

第十四次作业

张钧玮 2023K8009908003

课后练习42

■ 用单纯形法求解下列线性规划

$$(1) \quad \max Z = 3x_1 + 2x_2 - \frac{1}{8}x_3$$
$$\begin{cases} -x_1 + 2x_2 + 3x_3 \leq 4 \\ 4x_1 - 2x_3 \leq 12 \\ 3x_1 + 8x_2 + 4x_3 \leq 10 \\ x_j \geq 0, j = 1, 2, 3 \end{cases}$$

首先化为标准形式：

$$\max Z = 3x_1 + 2x_2 - \frac{1}{8}x_3$$
$$\begin{cases} -x_1 + 2x_2 + 2x_3 + x_4 = 4 \\ 4x_1 - 2x_3 + x_5 = 12 \\ 3x_1 + 8x_2 + 4x_3 + x_6 = 10 \\ x_j \geq 0, j = 1, 2, 3, 4, 5, 6 \end{cases}$$

系数矩阵为：

$$\begin{bmatrix} -1 & 2 & 2 & 1 & 0 & 0 \\ 4 & 0 & -2 & 0 & 1 & 0 \\ 3 & 8 & 4 & 0 & 0 & 1 \end{bmatrix}$$

选取 x_4, x_5, x_6 为基变量，解线性方程组得到： $X^0 = (0, 0, 0, 4, 12, 10)$ ，检验数为 $(3, 2, -\frac{1}{8}, 0, 0, 0)$ ，列表为：

$$\begin{cases} -x_1 + 2x_2 + 2x_3 + x_4 = 4 \\ 4x_1 - 2x_3 + x_5 = 12 \\ 3x_1 + 8x_2 + 4x_3 + x_6 = 10 \end{cases}$$

由于 λ_1 最大，故选择 x_1 进基。选择列表中的 x_1 列与 b 做比值，只选择正值，得到最小行为第二行 $\frac{12}{4=3}$ ，故选择 x_5 出基。

调整列表（ x_5 行除以4作为 x_1 行，同时消去 x_4 和 x_6 行的 x_1 ），得到：

$$\begin{cases} 2x_2 + \frac{3}{2}x_3 + x_4 + \frac{1}{4}x_5 = 7 \\ x_1 - \frac{1}{2}x_3 + \frac{1}{4}x_5 = 3 \\ 8x_2 + \frac{11}{2}x_3 - \frac{3}{4}x_5 + x_6 = 1 \end{cases}$$

重新求解线性方程组，得到校验数 $(0, 2, \frac{11}{8}, 0, -\frac{3}{4}, 0)$ 。校验数包含正数，因此还不是最优。选择 x_2 进基。选择列表中的 x_2 列与 b 做比值，只选择正值，得到最小行为第三行，故出基为 x_6 。

调整列表，得到：

$$\begin{cases} \frac{1}{8}x_3 + x_4 + \frac{7}{16}x_5 - \frac{1}{4}x_6 = \frac{27}{4} \\ x_1 - \frac{1}{2}x_3 + \frac{1}{4}x_5 = 3 \\ x_2 + \frac{11}{16}x_3 - \frac{3}{32}x_5 + \frac{1}{8}x_6 = \frac{1}{8} \end{cases}$$

重新求解线性方程组，得到基本可行解 $(3, \frac{1}{8}, 0, \frac{27}{4}, 0, 0)$ ，校验数为 $(0, 0, 0, 0, -\frac{9}{16}, -\frac{1}{4})$ ，全为负，故取得最优解。

课后练习43

■ 分别用大 M 法求解下列线性规划

$$\begin{aligned} (1) \quad \max Z &= 10x_1 - 5x_2 + x_3 \\ &\begin{cases} 5x_1 + 3x_2 + x_3 = 10 \\ -5x_1 + x_2 - 10x_3 \leq 15 \\ x_j \geq 0, j=1,2,3 \end{cases} \end{aligned} \quad \begin{aligned} (2) \quad \min Z &= 5x_1 - 6x_2 - 7x_3 \\ &\begin{cases} x_1 + 5x_2 - 3x_3 \geq 15 \\ 5x_1 - 6x_2 + 10x_3 \leq 20 \\ x_1 + x_2 + x_3 = 5 \\ x_j \geq 0, j=1,2,3 \end{cases} \end{aligned}$$

编写python程序使用大M法求解线性规划：

```
from dataclasses import dataclass
from typing import Any, Dict, List, Literal, Optional

import pulp

Sense = Literal["≤", "≥", "="]
ObjSense = Literal["max", "min"]

@dataclass
class LPStandard:
    # objective: c^T x
    c: List[float]
    # constraints: A x (≤, ≥, =) b
    A: List[List[float]]
    b: List[float]
    senses: List[Sense]
    obj_sense: ObjSense = "max"
    # optional variable bounds; default x ≥ 0
    lb: Optional[List[float]] = None
    ub: Optional[List[Optional[float]]] = None

def solve_big_m(lp: LPStandard, M: float = 1e6, msg: bool = False) → Dict[str, Any]:
    """
    Big-M method implemented on top of a generic LP solver (PuLP/CBC).
    Only the Big-M core logic is explicit: add artificial vars and penalize them in objective.

    Returns:
        dict with status, objective, x, artificials, feasible_under_big_m
    """
    n = len(lp.c)
    m = len(lp.A)
    assert len(lp.b) == m and len(lp.senses) == m
    assert all(len(row) == n for row in lp.A)

    # 1) Build PuLP model
    prob = pulp.LpProblem(
        "LP_BigM", pulp.LpMaximize if lp.obj_sense == "max" else pulp.LpMinimize
    )

    # 2) Decision variables x_j
    lb = lp.lb if lp.lb is not None else [0.0] * n
    ub = lp.ub if lp.ub is not None else [None] * n
```

```

x = [pulp.LpVariable(f"x{j + 1}", lowBound=lb[j], upBound=ub[j]) for j in range(n)]

# 3) For each constraint, add slack/surplus and (if needed) artificial variable
artificials = [] # track artificial vars for feasibility check and objective penalty
constraints = []

for i in range(m):
    expr = pulp.lpSum(lp.A[i][j] * x[j] for j in range(n))
    sense = lp.senses[i]
    rhs = lp.b[i]

    if sense == "≤":
        # Add slack s_i ≥ 0: expr + s_i = rhs (solver can keep it as ≤ directly)
        constraints.append(expr ≤ rhs)

    elif sense == "≥":
        # Add surplus and artificial:
        # expr - s_i + a_i = rhs, s_i ≥ 0, a_i ≥ 0
        s = pulp.LpVariable(f"s{i + 1}", lowBound=0)
        a = pulp.LpVariable(f"a{i + 1}", lowBound=0)
        artificials.append(a)
        constraints.append(expr - s + a == rhs)

    elif sense == "=":
        # Add artificial:
        # expr + a_i = rhs, a_i ≥ 0
        a = pulp.LpVariable(f"a{i + 1}", lowBound=0)
        artificials.append(a)
        constraints.append(expr + a == rhs)

    else:
        raise ValueError(f"Unknown sense: {sense}")

for cons in constraints:
    prob += cons

# 4) Big-M objective: original objective + M * sum(artificials)
base_obj = pulp.lpSum(lp.c[j] * x[j] for j in range(n))
art_sum = pulp.lpSum(artificials) if artificials else 0

if lp.obj_sense == "max":
    prob += base_obj - M * art_sum
else: # min
    prob += base_obj + M * art_sum

# 5) Solve
prob.solve(pulp.PULP_CBC_CMD(msg=msg))

status = pulp.LpStatus[prob.status]
x_val = [pulp.value(v) for v in x]
a_val = [pulp.value(a) for a in artificials]
obj_val = pulp.value(prob.objective)

# 6) Feasibility under Big-M: artificials should be ~0 at optimum (tolerance)
tol = 1e-7
feasible_under_big_m = all((v is not None and abs(v) ≤ tol) for v in a_val)

return {
    "status": status,
    "objective_with_penalty": obj_val,
    "x": x_val,
    "artificials": a_val,

```

```

        "feasible_under_big_m": feasible_under_big_m,
        "M": M,
    }

if __name__ == "__main__":
    lp = LPStandard(
        c=[10, -5, 1],
        A=[[5, 3, 1], [-5, 1, -10]],
        b=[10, 15],
        senses=["=", "<="],
        obj_sense="max",
    )
    res = solve_big_m(lp, M=1e6, msg=False)
    print(res)

```

(1)

```

{'status': 'Optimal', 'objective_with_penalty': 20.0, 'x': [2.0, 0.0, 0.0], 'artificials':
[0.0], 'feasible_under_big_m': True, 'M': 1000000.0}

```

最优解为(2,0,0)，此时 $Z_{max} = 20$

(2)

```

{'status': 'Optimal', 'objective_with_penalty': -31.25, 'x': [0.0, 3.75, 1.25], 'artificials':
[0.0, 0.0], 'feasible_under_big_m': True, 'M': 1000000.0}

```

最优解为(0,3.75,1.25)，此时 $Z_{min} = -31.25$